Lekcja 15. Przekazywanie obiektów jako parametrów funkcji

Przedmiot: Programowanie aplikacji Klasa: 4 technikum Zawód: technik programista (351406) Kwalifikacja: INF.04

Powiązanie z podstawą programową: INF.04.4 Programowanie obiektowe - uczeń stosuje zasady programowania obiektowego, definiuje klasy i metody oraz korzysta z obiektów jako składników programu.

1. Cele lekcji

• wyjaśnić problem kopiowania obiektów podczas przekazywania argumentów do funkcji;
• zastosować przekazywanie obiektów przez referencję i przez wskaźnik;
• odróżnić parametr wejściowy od wyjściowego;
• użyć const do ochrony obiektu przed modyfikacją;
• napisać funkcje pobierające i wyświetlające dane obiektu.

2. Dlaczego nie zawsze przekazujemy obiekt przez wartość?

Obiekt jako instancja klasy może zawierać wiele pól, obiekty zagnieżdżone oraz inne składniki. Jeśli funkcja otrzyma taki obiekt przez wartość, tworzona jest kopia całego obiektu. Przy większych obiektach oznacza to większe zużycie pamięci operacyjnej oraz dodatkowy czas potrzebny na kopiowanie.

Dlatego dla dużych obiektów parametry funkcji najczęściej definiuje się jako referencje albo wskaźniki. Dzięki temu funkcja pracuje na istniejącym obiekcie, a nie na jego kopii.

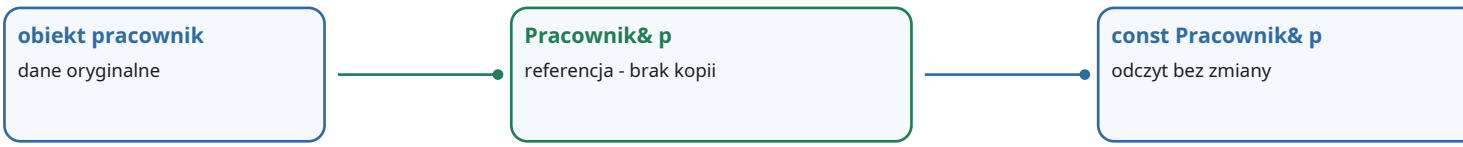
RAM (ang. Random Access Memory) - pamięć operacyjna, w której podczas działania programu przechowywane są m.in. zmienne, obiekty i kopie argumentów funkcji.

3. Parametry wejściowe i wyjściowe

Rodzaj parametru	Rola	Zalecany zapis
wejściowy	funkcja odczytuje dane obiektu	const Pracownik& p albo const Pracownik* p
wyjściowy	funkcja zmienia obiekt lub przekazuje wynik	Pracownik& p albo Pracownik* p

const oznacza, że przez dany parametr funkcja nie może zmienić przekazanego obiektu. Jest to szczególnie ważne przy parametrach wejściowych, które mają służyć tylko do odczytu.

4. Przekazywanie przez referencję - przykład 11.11



W wersji z referencją funkcja otrzymuje dostęp do istniejącego obiektu. Składnia użycia pól jest taka sama jak dla zwykłego obiektu, np. p.imie. Nie stosujemy operatora strzałkowego.

```
#include <iostream>
using namespace std;

class Pracownik {
public:
    string imie;
    string nazwisko;
};

Pracownik& pobierzDane(Pracownik& p) {
```

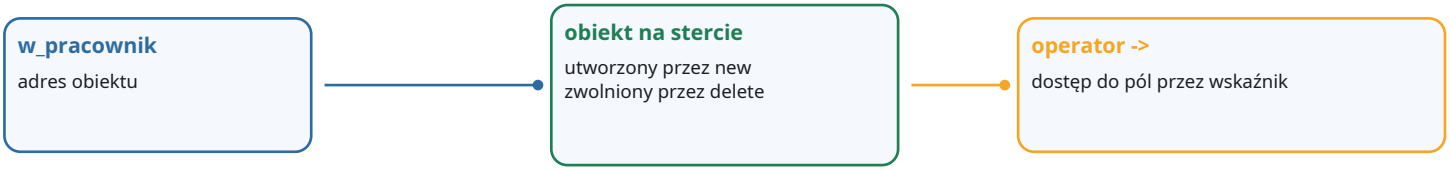
```
    cout << "Imię = "; cin >> p.imie;
    cout << "Nazwisko = "; cin >> p.nazwisko;
    return p;
}

void wyswietlDane(const Pracownik& p) {
    cout << "Imię: " << p.imie << endl;
    cout << "Nazwisko: " << p.nazwisko << endl;
}

int main() {
    Pracownik pracownik;
    pobierzDane(pracownik);
    wyswietlDane(pracownik);
    return 0;
}
```

Funkcja pobierzDane(Pracownik& p) może zmieniać pola obiektu. Funkcja wyswietlDane(const Pracownik& p) tylko odczytuje dane, dlatego parametr został zapisany jako const.

5. Przekazywanie przez wskaźnik - przykład 11.12



Wskaźnik przechowuje adres obiektu. Do pól i metod obiektu wskazywanego przez wskaźnik odwołujemy się operatorem strzałkowym ->. Jeśli obiekt utworzono dynamicznie operatorem new, po zakończeniu pracy należy zwolnić pamięć operatorem delete.

```
Pracownik* pobierzDane(Pracownik* w_p);
void wyswietlDane(const Pracownik* w_p);

int main() {
    Pracownik* w_pracownik = new Pracownik();
    pobierzDane(w_pracownik);
    wyswietlDane(w_pracownik);
    delete w_pracownik;
    return 0;
}

Pracownik* pobierzDane(Pracownik* w_p) {
    cout << "Imię = "; cin >> w_p->imie;
    cout << "Nazwisko = "; cin >> w_p->nazwisko;
    return w_p;
}
```

nullptr - wartość wskaźnika oznaczająca, że nie wskazuje on na żaden poprawny obiekt. Przed użyciem wskaźnika warto sprawdzić, czy nie ma wartości nullptr.

6. Referencja a wskaźnik - szybkie porównanie

Cecha	Referencja	Wskaźnik
Sposób zapisu dostępu do pola	p.imie	p->imie
Może oznaczać brak obiektu	nie w typowym użyciu	tak, przez nullptr
Czytelność kodu	zwykle większa	wymaga ostrożności
Typowe zastosowanie	obiekt istnieje i ma być przekazany bez kopii	obiekt może być dynamiczny lub opcjonalny

7. Najważniejsze zasady

• Nie przekazuj dużych obiektów przez wartość, jeśli nie potrzebujesz kopii.
• Jeśli funkcja tylko odczytuje obiekt, użyj const referencji lub const wskaźnika.

- Jeśli funkcja ma zmienić obiekt, użyj referencji lub wskaźnika bez const.
- Dla obiektu i referencji używaj operatora kropki ., a dla wskaźnika operatora strzałkowego ->.
- Pamiętaj o zwolnieniu pamięci, jeśli obiekt został utworzony przez new.